

Redux Toolkit - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

REDUX TOOLKIT — MASTER NOTES	4
A Complete Reference Book for Interviews, Production Work & System Understanding	4
TOPIC 1: STORE	4
1. Introduction	4
1.1 What is it, and why does it exist?	4
1.2 The exact problem it solves — Beyond What Context API (React section) Handles	4
2. FULL STEPS: Setting Up Redux Toolkit	4
STEP 1 — Install	4
STEP 2 — Create the Store	5
STEP 3 — Provide the Store to Your React App	5
3. Real Life Analogy	5
4. Store vs Context API — When to Use Which	5
5. Frequently Asked Interview Questions	5
TOPIC 2: SLICE	6
1. Introduction	6
2. Code Example — createSlice	6
3. The "Magic" Behind createSlice — Immer	6
4. Real Life Analogy	7
5. Frequently Asked Interview Questions	7
TOPIC 3: REDUCERS	7
1. Introduction	7
2. What Makes a Function a "Pure" Reducer (Critical Concept)	7
3. The combineReducers Concept (What configureStore Does Automatically)	8
4. Real Life Analogy	8
5. Frequently Asked Interview Questions	8
TOPIC 4: ACTIONS	8
1. Introduction	8

2. Code Example — Dispatching Actions From a Component	9
3. Anatomy of an Action	9
4. useSelector — Reading State Efficiently	9
5. Real Life Analogy	9
6. Frequently Asked Interview Questions	9

TOPIC 5: ASYNCTHUNK 10

1. Introduction	10
2. Code Example — Fetching Employees From Your Spring Boot API	10
3. Using the Thunk From a Component	11
4. Real Life Analogy	11
5. Frequently Asked Interview Questions	11

TOPIC 6: MIDDLEWARE 11

1. Introduction	11
2. How Middleware Fits Into the Flow	12
3. Code Example — A Simple Logging Middleware	12
4. Real Life Analogy	12
5. Frequently Asked Interview Questions	12

TOPIC 7: RTK QUERY (BASICS) 13

1. Introduction	13
2. Code Example — Defining an API Slice	13
3. Using the Auto-Generated Hooks — Dramatically Less Code Than Thunks	13
4. Register the API Slice With the Store	14
5. RTK Query vs Manual createAsyncThunk — When to Use Which	14
6. Real Life Analogy	14
7. Frequently Asked Interview Questions	14
8. Revision Notes (Entire Redux Toolkit Section Consolidated)	15
9. Homework (Covers Entire Redux Toolkit Section)	15
FINAL SUMMARY — REDUX TOOLKIT COMPLETE	15

REDUX TOOLKIT — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: STORE

1. Introduction

1.1 What is it, and why does it exist?

Redux is a STATE MANAGEMENT library — **Redux Toolkit (RTK)** is the OFFICIAL, modern, recommended way to use Redux today, designed SPECIFICALLY to eliminate the notoriously verbose boilerplate that "plain" Redux required. The **Store** is Redux's SINGLE, CENTRALIZED container holding your ENTIRE application's shared state — one object, in one place, that any component can read from or update through well-defined rules.

1.2 The exact problem it solves — Beyond What Context API (React section) Handles

Recall the React section's Context API, which solved "prop drilling" for SIMPLE shared data. Redux exists for a DIFFERENT, LARGER-SCALE problem:

CONTEXT API STRUGGLES WHEN:

- State is updated VERY FREQUENTLY (every Context consumer re-renders on ANY change, becoming a real performance problem at scale - Context wasn't DESIGNED as a general-purpose state management solution)
- You need PREDICTABLE, TRACEABLE state changes across a LARGE application with MANY different pieces of state, updated from MANY different places
- You want powerful DEVELOPER TOOLS - time-travel debugging, seeing the EXACT sequence of every state change that ever happened, for troubleshooting

REDUX TOOLKIT SOLVES THIS BY:

- Centralizing ALL application state in ONE store
- Making EVERY state change happen through a STRICT, PREDICTABLE pattern (dispatch an ACTION -> a REDUCER computes the NEW state) - never direct mutation
- Providing excellent DevTools showing the complete history of every action/state change

2. FULL STEPS: Setting Up Redux Toolkit

STEP 1 — Install

```
npm install @reduxjs/toolkit react-redux
```

STEP 2 — Create the Store

```
// store.js
import { configureStore } from "@reduxjs/toolkit";
import employeeReducer from "../employeeSlice"; // from a "slice", covered in Topic 2

export const store = configureStore({
  reducer: {
    employees: employeeReducer, // the store's SHAPE: { employees: {...} }
  },
});
```

STEP 3 — Provide the Store to Your React App

```
// index.js
import { Provider } from "react-redux";
import { store } from "../store";

<Provider store={store}> { /* makes the store available to ANY component in the tree, without prop drilling */ }
<App />
</Provider>
```

3. Real Life Analogy

- **Restaurant:** The Store is like a restaurant chain's SINGLE, centralized master ledger/database — instead of every individual location keeping its OWN separate, potentially inconsistent records, ALL locations read from and update the SAME centralized source of truth, through STRICT, standardized procedures (never just scribbling directly on the ledger) — ensuring the entire chain's data stays consistent and auditable.

4. Store vs Context API — When to Use Which

Aspect	Context API	Redux Toolkit
Best for	Simple, INFREQUENTLY changing shared data (current theme, logged-in user)	Complex, FREQUENTLY changing state across a large app
Performance at scale	Can degrade (every consumer re-renders on ANY change)	Optimized — components only re-render when their SPECIFIC piece of state changes
DevTools/debugging	Minimal	Excellent (Redux DevTools — action history, time-travel debugging)
Setup complexity	Very low	Higher (worth it for genuinely complex apps)

5. Frequently Asked Interview Questions

- 1 **What is the Redux Store?** A single, centralized container holding an entire application's shared state, updated only through a strict, predictable dispatch-action-reducer pattern.
- 2 **What problem does Redux Toolkit solve beyond what Context API already handles?** Managing FREQUENTLY-changing, complex state at scale, with better performance characteristics and powerful debugging tools (action history, time-travel debugging) — Context API wasn't designed as a general-purpose, high-frequency state management solution.

- 3 **What does** `<Provider store={store}>` **do?** Makes the Redux store available to any component in the React tree, without needing to manually pass it down through props.
-

TOPIC 2: SLICE

1. Introduction

What is it? A "slice" is Redux Toolkit's core organizational unit — it bundles TOGETHER the initial state, reducers (Topic 3), and auto-generated actions (Topic 4) for ONE specific piece of your application's state (e.g., "everything related to employees"), in ONE file, eliminating the SEPARATE action-types/action-creators/reducer files that "plain" (pre-Toolkit) Redux notoriously required.

2. Code Example — `createSlice`

```
// employeeSlice.js
import { createSlice } from "@reduxjs/toolkit";

const employeeSlice = createSlice({
  name: "employees", // used as a PREFIX for auto-generated action types (e.g., "employees/addEmployee")
  initialState: {
    list: [],
    loading: false,
  },
  reducers: {
    addEmployee: (state, action) => {
      state.list.push(action.payload); // looks like MUTATION, but Immer (used internally) makes it SAFE!
    },
    removeEmployee: (state, action) => {
      state.list = state.list.filter(emp => emp.id !== action.payload);
    },
    setLoading: (state, action) => {
      state.loading = action.payload;
    },
  },
});

export const { addEmployee, removeEmployee, setLoading } = employeeSlice.actions; // auto-generated ACTIONS!
export default employeeSlice.reducer; // the REDUCER itself
```

3. The "Magic" Behind `createSlice` — Immer

`createSlice` uses a library called **Immer** internally, which lets you write code that LOOKS like direct mutation (`state.list.push(...)`) but is ACTUALLY producing a brand-new, immutable state object behind the scenes — this is a DELIBERATE ergonomic improvement Redux Toolkit provides over "plain" Redux, which STRICTLY required manually returning NEW objects/arrays everywhere (`return {...state, list: [...state.list, newItem]}`), a common source of subtle bugs.

4. Real Life Analogy

- **Restaurant:** A slice is like a SINGLE, complete recipe binder for ONE specific menu category ("Appetizers") — containing the STARTING ingredients list (initial state), the STANDARDIZED preparation steps (reducers), and the LABELED order tickets kitchen staff use to trigger each step (actions) — ALL organized together in ONE binder, instead of scattered across separate files for ingredients, steps, and tickets.

5. Frequently Asked Interview Questions

- 1 **What is a Redux Toolkit "slice"?** A bundle of initial state, reducers, and auto-generated actions for ONE piece of application state, defined together via `createSlice`.
- 2 **What library does `createSlice` use internally to allow "mutating" syntax safely?** Immer — it lets you write code that LOOKS like direct mutation, while actually producing a new immutable state object behind the scenes.
- 3 **What problem did `createSlice` solve compared to "plain" (pre-Toolkit) Redux?** Eliminated the need for separate action-type constants, action-creator functions, and manually-written immutable reducer logic scattered across multiple files — all consolidated into one slice definition.

TOPIC 3: REDUCERS

1. Introduction

What is it? A reducer is a PURE FUNCTION that takes the CURRENT state and an ACTION, and returns the NEW state — the actual LOGIC deciding HOW state changes in response to each action, defined inside a slice's `reducers` object (Topic 2).

2. What Makes a Function a "Pure" Reducer (Critical Concept)

```
// A PURE function: SAME inputs ALWAYS produce the SAME output, with NO side effects
// (no API calls, no random values, no modifying anything OUTSIDE its own scope)

reducers: {
  addEmployee: (state, action) => {
    state.list.push(action.payload); // ONLY computes new state from (state, action) - nothing else!
  },
}
```

This is EXACTLY why side effects (API calls) do NOT belong directly inside a reducer — they belong in `AsyncThunk` (Topic 5) instead, keeping reducers predictable, easily testable, and fully compatible with Redux's time-travel debugging (which relies on being able to safely RE-RUN reducers to reconstruct past states).

3. The `combineReducers` Concept (What `configureStore` Does Automatically)

```
// configureStore's `reducer: { employees: employeeReducer, departments: departmentReducer }`  
// is CONCEPTUALLY equivalent to:  
const rootReducer = combineReducers({  
  employees: employeeReducer,  
  departments: departmentReducer,  
});  
// producing a store SHAPE like: { employees: {...}, departments: {...} }
```

4. Real Life Analogy

- **Bank:** A reducer is like a bank's STANDARDIZED, deterministic transaction-processing RULE — given the EXACT SAME starting account balance and the EXACT SAME transaction request, it ALWAYS produces the EXACT SAME resulting balance, EVERY single time, with NO reliance on anything else (no checking today's weather, no random behavior) — this predictability is EXACTLY what makes bank audits (and Redux's time-travel debugging) possible.

5. Frequently Asked Interview Questions

- 1 **What is a reducer in Redux?** A pure function that takes the current state and an action, and returns the new state.
- 2 **Why must a reducer be a PURE function (no side effects)?** To keep state changes predictable, testable, and compatible with Redux's debugging tools (like time-travel debugging), which rely on being able to safely re-run reducers to reconstruct past application states.
- 3 **Where should API calls/side effects go instead of directly inside a reducer?** Inside an `AsyncThunk` (Topic 5), keeping the reducer itself pure.

TOPIC 4: ACTIONS

1. Introduction

What is it? An action is a PLAIN JAVASCRIPT OBJECT describing WHAT HAPPENED — the ONLY way to trigger a state change in Redux; you never modify the store directly, you `dispatch()` an action, and the appropriate reducer decides how to respond to it.

2. Code Example — Dispatching Actions From a Component

```
import { useDispatch, useSelector } from "react-redux";
import { addEmployee, removeEmployee } from "../employeeSlice";

function EmployeeManager() {
  const dispatch = useDispatch(); // gives access to dispatch actions
  const employees = useSelector(state => state.employees.list); // READS a piece of state from the store

  const handleAdd = () => {
    dispatch(addEmployee({ id: 1, firstName: "Alice", salary: 75000 })); // action = { type:
    "employees/addEmployee", payload: {...} }
  };

  return (
    <div>
      {employees.map(emp => (
        <div key={emp.id}>
          {emp.firstName}
          <button onClick={() => dispatch(removeEmployee(emp.id))}>Remove</button>
        </div>
      ))}
      <button onClick={handleAdd}>Add Employee</button>
    </div>
  );
}
```

3. Anatomy of an Action

```
{
  type: "employees/addEmployee", // auto-generated from the SLICE NAME + REDUCER NAME (Topic 2)
  payload: { id: 1, firstName: "Alice", salary: 75000 } // the DATA carried along with this action
}
```

4. `useSelector` — Reading State Efficiently

```
const employees = useSelector(state => state.employees.list);
```

`useSelector` is smart — the component using it ONLY re-renders when the SPECIFIC piece of state it selects ACTUALLY changes, not on EVERY store update — this is precisely the performance advantage Redux has over Context API at scale (recap of Topic 1).

5. Real Life Analogy

- **Restaurant:** An action is like a STANDARDIZED order ticket ("ADD_DISH", with details about which dish and table) submitted to the kitchen — the kitchen (reducer) reads the ticket and decides EXACTLY how to update the current order status accordingly; the waiter (component) never walks into the kitchen and directly rearranges the cooking process themselves.

6. Frequently Asked Interview Questions

- 1 **What is a Redux action?** A plain JavaScript object describing what happened, containing a `type` and typically a `payload`, dispatched to trigger a state change.

- 2 **How do actions relate to reducers?** Dispatching an action triggers the reducer, which reads the action's `type/payload` and computes the corresponding new state.
 - 3 **What does `useSelector` do, and why is it more efficient than a naive "re-render on any store change" approach?** It reads a SPECIFIC piece of state from the store, only re-rendering the component when THAT specific piece actually changes — not on every unrelated store update.
-
-

TOPIC 5: ASYNCTHUNK

1. Introduction

What is it? Since reducers MUST be pure (no side effects, Topic 3), `createAsyncThunk` provides the OFFICIAL way to handle ASYNCHRONOUS operations (API calls to your Spring Boot backend) in Redux Toolkit — automatically dispatching `pending/fulfilled/rejected` actions as the async operation progresses, which your slice can then react to.

2. Code Example — Fetching Employees From Your Spring Boot API

```
// employeeSlice.js
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchEmployees = createAsyncThunk(
  "employees/fetchEmployees", // action type PREFIX
  async () => {
    const response = await fetch("http://localhost:8080/api/employees");
    return response.json(); // this becomes the action's payload on SUCCESS
  }
);

const employeeSlice = createSlice({
  name: "employees",
  initialState: { list: [], loading: false, error: null },
  reducers: { /* ... synchronous reducers from Topic 2 ... */ },
  extraReducers: (builder) => { // handles the THREE auto-generated lifecycle actions
    builder
      .addCase(fetchEmployees.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchEmployees.fulfilled, (state, action) => {
        state.loading = false;
        state.list = action.payload; // action.payload = whatever the thunk RETURNED
      })
      .addCase(fetchEmployees.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  },
});

export default employeeSlice.reducer;
```

3. Using the Thunk From a Component

```
function EmployeeList() {
  const dispatch = useDispatch();
  const { list, loading, error } = useSelector(state => state.employees);

  useEffect(() => {
    dispatch(fetchEmployees()); // DISPATCHING a thunk kicks off the async flow automatically
  }, [dispatch]);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return list.map(emp => <div key={emp.id}>{emp.firstName}</div>);
}
```

4. Real Life Analogy

- **Restaurant:** `createAsyncThunk` is like a waiter placing a SPECIAL SUPPLIER order that takes TIME to fulfill — the SYSTEM automatically posts a "Order Placed" notice (pending), and LATER, EITHER a "Delivery Arrived" notice (fulfilled, with the actual goods) OR a "Delivery Failed" notice (rejected, with a reason) — the kitchen (your slice's `extraReducers`) reacts appropriately to WHICHEVER outcome actually occurs.

5. Frequently Asked Interview Questions

- 1 **Why can't you just call `fetch()` directly inside a reducer?** Reducers must be PURE functions (no side effects, no async operations) — `createAsyncThunk` is the official mechanism for handling async operations while keeping the actual reducer logic pure.
- 2 **What three action types does `createAsyncThunk` automatically generate/dispatch?** `pending`, `fulfilled`, and `rejected`, corresponding to the async operation's lifecycle.
- 3 **Where do you handle a thunk's pending/fulfilled/rejected actions in a slice?** In the `extraReducers` builder callback, using `builder.addCase(...)`.

TOPIC 6: MIDDLEWARE

1. Introduction

What is it? Middleware sits BETWEEN dispatching an action and it actually REACHING the reducer — intercepting EVERY action, letting you run additional logic (logging, handling async thunks under the hood, crash reporting) — directly analogous to a Servlet Filter (Advanced Java) or Spring's AOP-based interceptors (`@Around` advice), just applied to Redux's action-dispatch pipeline instead.

2. How Middleware Fits Into the Flow

```
dispatch(action)
|
v
+-----+ +-----+ +-----+
| Middleware 1 | -> | Middleware 2 | -> | REDUCER |
| (e.g., logging) | | (e.g., thunk | | (computes new |
| | | handling) | | state) |
+-----+ +-----+ +-----+
```

3. Code Example — A Simple Logging Middleware

```
const loggerMiddleware = (store) => (next) => (action) => {
  console.log("Dispatching:", action.type);
  console.log("State BEFORE:", store.getState());
  const result = next(action); // pass the action ALONG to the next middleware/reducer
  console.log("State AFTER:", store.getState());
  return result;
};

const store = configureStore({
  reducer: { employees: employeeReducer },
  middleware: (getDefaultMiddleware) => getDefaultMiddleware().concat(loggerMiddleware),
});
```

Note: `createAsyncThunk` (Topic 5) ITSELF is implemented as a piece of middleware, ALREADY included by default in `configureStore` — this is exactly HOW dispatching a thunk (which isn't a plain action object) is intercepted and correctly handled before it would otherwise confuse a reducer expecting a plain `{type, payload}` object.

4. Real Life Analogy

- **Restaurant:** Middleware is like a series of CHECKPOINTS every order ticket passes through on its way to the kitchen — one checkpoint LOGS every order for record-keeping, another checkpoint SPECIAL-HANDLES complex, multi-step catering orders (thunks) before they reach the standard kitchen line (reducer) — exactly the SAME layered-interception concept as a Servlet Filter chain.

5. Frequently Asked Interview Questions

- 1 **What is Redux middleware?** Code that intercepts EVERY dispatched action before it reaches the reducer, enabling additional logic like logging or handling async operations.
- 2 **What's a real-world parallel to Redux middleware from earlier in this course?** A Servlet Filter (Advanced Java) or Spring AOP's `@Around` advice — both intercept a "pipeline" of calls, running logic before/after the actual target logic executes.
- 3 **Is `createAsyncThunk` itself implemented as middleware?** Yes — it's included by default in `configureStore`'s middleware chain, which is exactly how dispatching a thunk function (rather than a plain action object) is properly intercepted and handled.

TOPIC 7: RTK QUERY (BASICS)

1. Introduction

What is it? RTK Query is Redux Toolkit's BUILT-IN DATA-FETCHING solution — it goes a step BEYOND `createAsyncThunk` (Topic 5), AUTOMATICALLY generating hooks for calling your API, managing loading/error states, and CACHING results, eliminating even MORE boilerplate than manually writing thunks + `extraReducers` yourself.

2. Code Example — Defining an API Slice

```
// employeeApi.js
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

export const employeeApi = createApi({
  reducerPath: "employeeApi",
  baseQuery: fetchBaseQuery({ baseUrl: "http://localhost:8080/api" }),
  endpoints: (builder) => ({
    getEmployees: builder.query({
      query: () => "/employees", // GET /api/employees
    }),
    addEmployee: builder.mutation({
      query: (newEmployee) => ({
        url: "/employees",
        method: "POST",
        body: newEmployee,
      }),
    }),
  }),
});

export const { useGetEmployeesQuery, useAddEmployeeMutation } = employeeApi; // AUTO-GENERATED hooks!
```

3. Using the Auto-Generated Hooks — Dramatically Less Code Than Thunks

```
function EmployeeList() {
  const { data: employees, isLoading, error } = useGetEmployeesQuery(); // ONE line replaces the ENTIRE
  // thunk + extraReducers setup!
  const [addEmployee] = useAddEmployeeMutation();

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error loading employees</p>;

  return (
    <div>
      {employees.map(emp => <div key={emp.id}>{emp.firstName}</div>)}
      <button onClick={() => addEmployee({ firstName: "New Employee", salary: 60000 })}>Add</button>
    </div>
  );
}
```

4. Register the API Slice With the Store

```
export const store = configureStore({
  reducer: {
    [employeeApi.reducerPath]: employeeApi.reducer,
  },
  middleware: (getDefaultMiddleware) => getDefaultMiddleware().concat(employeeApi.middleware),
});
```

5. RTK Query vs Manual `createAsyncThunk` — When to Use Which

Aspect	<code>createAsyncThunk</code> (Topic 5)	RTK Query
Boilerplate	Manual thunk + <code>extraReducers</code> for EACH endpoint	Auto-generated hooks — dramatically less code
Caching	Manual (you manage it yourself)	AUTOMATIC — repeated calls for the same data are cached and deduplicated
Loading/error state	Manually tracked in your slice	AUTOMATICALLY provided by the generated hook (<code>isLoading</code> , <code>error</code>)
Best for	Complex, highly custom async logic	The VAST MAJORITY of standard "call my REST API" scenarios

6. Real Life Analogy

- **Restaurant:** `createAsyncThunk` is like a waiter manually tracking EVERY supplier order's status themselves on a personal notepad. RTK Query is like the restaurant having a FULLY AUTOMATED supplier-ordering system that tracks status, CACHES recent supplier responses (so it doesn't ask the SAME supplier the SAME question twice unnecessarily), and instantly tells ANY staff member the current status — dramatically less manual bookkeeping for the exact same underlying need.

7. Frequently Asked Interview Questions

- 1 **What is RTK Query?** Redux Toolkit's built-in data-fetching solution, automatically generating hooks for API calls with built-in loading/error state and caching, reducing boilerplate beyond even `createAsyncThunk`.
- 2 **What does RTK Query provide automatically that you'd otherwise manage yourself with `createAsyncThunk`?** Caching of results (avoiding redundant duplicate requests), and automatic loading/error state tracking, without manually writing `extraReducers` for each endpoint.
- 3 **When would you still use `createAsyncThunk` directly instead of RTK Query?** For complex, highly custom async logic that doesn't fit RTK Query's standard request/cache model.

8. Revision Notes (Entire Redux Toolkit Section Consolidated)

REDUX TOOLKIT — CHEAT SHEET

=====

SOLVES: state management at a scale beyond what Context API handles well - frequent updates, complex apps, powerful DevTools (action history, time-travel debugging)

STORE: ONE centralized state container, `configureStore({reducer: {...}})`, `<Provider store={store}>` makes it available app-wide

SLICE: `createSlice({name, initialState, reducers})` - bundles state + reducers + auto-generated actions in ONE place (uses Immer internally - "mutating" syntax is actually SAFE, produces new immutable state)

REDUCER: PURE function (state, action) -> newState - NO side effects/API calls allowed (that's what AsyncThunk is for)

ACTION: plain object {type, payload}, the ONLY way to trigger a state change - `dispatch(action)`, `useSelector(state => ...)` to READ state efficiently (only re-renders on THAT piece changing)

ASYNCThunk: `createAsyncThunk` for API calls - auto-dispatches pending/fulfilled/rejected, handled in a slice's `extraReducers`

MIDDLEWARE: intercepts EVERY action before the reducer (like a Servlet Filter / Spring AOP @Around) - `createAsyncThunk` itself is built as middleware, included by default in `configureStore`

RTK QUERY: `createApi` + `builder.query()/builder.mutation()` -> AUTO-GENERATED hooks (`useGetXQuery`, `useAddXMutation`) with BUILT-IN loading/error state AND caching - the modern, recommended default for standard REST API calls

9. Homework (Covers Entire Redux Toolkit Section)

- 1 Set up a Redux store with an `employeeSlice` managing a list of employees, dispatching `addEmployee/removeEmployee` actions from a component.
- 2 Add a `createAsyncThunk` fetching employees from your Spring Boot API, handling `pending/fulfilled/rejected` in `extraReducers`.
- 3 Write a simple logging middleware and confirm it logs every dispatched action's type and the state before/after.
- 4 Rewrite the same employee-fetching logic using RTK Query's `createApi`, comparing the total amount of code needed against your manual thunk-based approach.

FINAL SUMMARY — REDUX TOOLKIT COMPLETE

This concludes the **Redux Toolkit** section — 7 topics:

- 1 **Store** — the centralized state container, and when it's warranted over Context API.
- 2 **Slice** — bundling state/reducers/actions together, powered by Immer.
- 3 **Reducers** — pure functions, why side effects don't belong there.
- 4 **Actions** — the dispatch/action/reducer flow, efficient reading via `useSelector`.

- 5 **AsyncThunk** — the official way to handle API calls in Redux.
- 6 **Middleware** — the action-interception pipeline, the Servlet Filter/AOP parallel.
- 7 **RTK Query** — the modern, auto-generated, cached data-fetching layer.

You now have the tools to manage complex, scalable frontend state — completing the modern React/TypeScript/Redux stack that consumes the Spring Boot APIs built throughout this entire course. Next: CI/CD, covering how all of this (frontend and backend) gets automatically tested and deployed.

(END OF REDUX TOOLKIT MASTER NOTES — ready to proceed to CI/CD whenever you say "Next Topic.")